

Vel Tech Group of Institutions

Name: GOPICHAND KAKIVAI

Email: vtu29748@veltech.edu.in

Roll no: vtu29748

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L3

VTU_DAA_Week 8_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 29.5

Section 1 : Coding

1. Problem Statement

Sophia, a robotics engineer, is designing a task execution system where tasks have dependencies. Each task is represented as a node in a directed acyclic graph (DAG), and a directed edge $(u \rightarrow v)$ means task u must be completed before task v .

To determine the correct execution order, Sophia needs to perform a topological sort on the graph.

Your task is to help Sophia implement topological sorting and print the order in which tasks should be executed.

Input Format

The first line contains an integer N , representing the number of vertices (tasks)

in the DAG.

The second line contains an integer E, representing the number of directed edges (dependencies) in the graph.

The next E lines each contain two space-separated integers:

- src The starting vertex of a directed edge.
- dest The ending vertex of a directed edge.

Output Format

The output prints N space-separated integers in a single line, representing the vertices in topological order.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

6

5 2

5 0

4 0

4 1

2 3

3 1

Output: 4 5 0 2 3 1

Answer

-

Status : Skipped

Marks : 0/10

2. Problem Statement

Write a program to find the topological order of vertices in a directed acyclic graph (DAG) represented as an adjacency list. This involves establishing a linear arrangement of vertices such that, for every directed edge (u, v), vertex u precedes vertex v in the sequence.

Your task specifically requires printing the topological order in the reverse order of their respective end times obtained through a depth-first search (DFS) traversal.

Input Format

The first line consists of an integer N, representing the number of vertices.

The second line consists of an integer E, representing the number of directed edges.

The next E lines each contain two space-separated integers x and y, representing a directed edge from vertex x to vertex y.

Output Format

The output prints Topological Order: followed by N space-separated integers representing the vertices (0-indexed) in topological order, in reverse order of DFS end times.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

5

1 2

2 3

3 4

3 5

4 5

Output: Topological Order: 1 2 3 4 5

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {  
    int dest;  
    struct Node* next;
```

```
};
```

```
struct Graph {  
    int numVertices;  
    struct Node** adjLists;  
    int* visited;  
};
```

```
struct Node* createNode(int v) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    newNode->dest = v;  
    newNode->next = NULL;  
    return newNode;  
}
```

```
struct Graph* createGraph(int vertices) {  
    struct Graph* graph = (struct Graph*)malloc(sizeof(struct Graph));  
    graph->numVertices = vertices;  
    graph->adjLists = (struct Node**)malloc((vertices + 1) * sizeof(struct Node*));  
    graph->visited = (int*)malloc((vertices + 1) * sizeof(int));  
  
    for (int i = 0; i <= vertices; i++) {  
        graph->adjLists[i] = NULL;  
        graph->visited[i] = 0;  
    }  
    return graph;  
}
```

```
void addEdge(struct Graph* graph, int src, int dest) {  
    struct Node* newNode = createNode(dest);  
    newNode->next = graph->adjLists[src];  
    graph->adjLists[src] = newNode;  
}
```

```
void dfs(struct Graph* graph, int vertex, int* stack, int* top) {  
    graph->visited[vertex] = 1;  
    struct Node* temp = graph->adjLists[vertex];  
  
    while (temp) {  
        int adjVertex = temp->dest;  
        if (!graph->visited[adjVertex]) {  
            dfs(graph, adjVertex, stack, top);  
        }  
        temp = temp->next;  
    }  
}
```

```

    }
    temp = temp->next;
}
stack[( *top)++] = vertex;
}

int main() {
    int N, E;
    if (scanf("%d %d", &N, &E) != 2) return 0;

    struct Graph* graph = createGraph(N);

    for (int i = 0; i < E; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        addEdge(graph, u, v);
    }

    int* stack = (int*)malloc((N + 1) * sizeof(int));
    int top = 0;

    for (int i = 1; i <= N; i++) {
        if (!graph->visited[i]) {
            dfs(graph, i, stack, &top);
        }
    }

    printf("Topological Order:");
    for (int i = top - 1; i >= 0; i--) {
        printf(" %d", stack[i]);
    }
    printf("\n");

    return 0;
}

```

Status : Partially correct

Marks : 7.5/10

3. Problem Statement

Given a total of N tasks labeled from 0 to $N-1$. Some tasks may have prerequisites. A prerequisite pair $[u, v]$ means task u must be completed before task v . Given the total number of tasks and a list of prerequisite pairs, return the ordering of tasks you should complete to finish all tasks. There may be multiple correct orders – return any one of them. If it is impossible to finish all tasks (due to a cycle), print nothing.

Examples

Input: $N=2$, Prerequisite_Pairs=[[1, 0]]

Output: [0, 1]

Explanation: There are a total of 2 tasks to pick. To pick task 1 you should have finished task 0. So the correct task order is [0, 1] .

Input: $N=4$, Prerequisite_Pairs=[[1, 0], [2, 0], [3, 1], [3, 2]]

Output: [0, 1, 2, 3] or [0, 2, 1, 3]

Explanation: There are a total of 4 tasks to pick. To pick task 3 you should have finished both tasks 1 and 2. Both tasks 1 and 2 should be pick after you finished task 0. So one correct task order is [0, 1, 2, 3]. Another correct ordering is [0, 2, 1, 3].

Input Format

The first line of input consists of an integer n , representing the number of tasks.

The second line consists of an integer p , representing the number of prerequisite pairs.

The next p lines each consist of two space-separated integers u and v , representing that task u must be completed before task v .

Output Format

The output prints the n tasks as space-separated integers on a single line, representing a valid task execution order. If no valid ordering exists, print nothing.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

0

Output: 0 1 2

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {
```

```
    int N, P;
```

```
    if (scanf("%d", &N) != 1) return 0;
```

```
    if (scanf("%d", &P) != 1) return 0;
```

```
    int adj[101][101] = {0};
```

```
    int indegree[101] = {0};
```

```
    int queue[101];
```

```
    int result[101];
```

```
    int front = 0, rear = 0, count = 0;
```

```
    for (int i = 0; i < P; i++) {
```

```
        int u, v;
```

```
        if (scanf("%d %d", &u, &v) == 2) {
```

```
            // Task u must be completed before task v (u -> v)
```

```
            if (adj[u][v] == 0) {
```

```
                adj[u][v] = 1;
```

```
                indegree[v]++;
```

```
            }
```

```
        }
```

```
    }
```

```
    for (int i = 0; i < N; i++) {
```

```
        if (indegree[i] == 0) {
```

```
            queue[rear++] = i;
```

```
        }
```

```
    }
```

```
    while (front < rear) {
```

```
        int u = queue[front++];
```

```

        result[count++] = u;
        for (int v = 0; v < N; v++) {
            if (adj[u][v]) {
                indegree[v]--;
                if (indegree[v] == 0) {
                    queue[rear++] = v;
                }
            }
        }
    }
}

if (count == N) {
    for (int i = 0; i < N; i++) {
        printf("%d", result[i]);
        if (i < N - 1) printf(" ");
    }
    printf("\n");
}

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Raj is given a weighted directed graph with N vertices and the weights of the edges between the vertices.

His task is to implement Dijkstra's algorithm to find the shortest distance from a given source vertex to all other vertices in the graph.

Input Format

The first line contains an integer N, the number of vertices in the graph.

The next N lines each contain N space-separated integers, representing the adjacency matrix of the graph. The value at the i-th row and j-th column represents the weight of the edge from vertex i to vertex j. A value of 0 indicates

no direct edge between the vertices.

The last line contains an integer src, representing the source vertex.

Output Format

The first line of output prints the heading: Vertex Distance from Source

The next N lines each print the vertex number and its shortest distance from the source vertex, separated by two tab characters. Vertices are numbered from 0 to N-1.

Refer to the sample output for formatting specifications

Sample Test Case

Input: 4

0 2 0 5

2 0 4 0

0 4 0 1

5 0 1 0

0

Output: Vertex Distance from Source

0 0

1 2

2 6

3 5

Answer

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
#include <stdbool.h>
```

```
#define MAX 100
```

```
int findMinDistance(int dist[], bool visited[], int N) {  
    int min = INT_MAX, min_index;
```

```
    for (int v = 0; v < N; v++) {  
        if (visited[v] == false && dist[v] <= min) {  
            min = dist[v];
```

```

        min_index = v;
    }
}

return min_index;
}

void dijkstra(int graph[MAX][MAX], int src, int N) {
    int dist[MAX];
    bool visited[MAX];

    for (int i = 0; i < N; i++) {
        dist[i] = INT_MAX;
        visited[i] = false;
    }

    dist[src] = 0;

    for (int count = 0; count < N - 1; count++) {
        int u = findMinDistance(dist, visited, N);
        visited[u] = true;

        for (int v = 0; v < N; v++) {
            if (!visited[v] && graph[u][v] && dist[u] != INT_MAX
                && dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }

    printf("Vertex Distance from Source\n");
    for (int i = 0; i < N; i++) {
        printf("%d\t\t%d\n", i, dist[i]);
    }
}

```

```

int main() {
    int N;
    if (scanf("%d", &N) != 1) return 0;

    int graph[MAX][MAX];
    for (int i = 0; i < N; i++) {

```

```

    for (int j = 0; j < N; j++) {
        scanf("%d", &graph[i][j]);
    }
}

int src;
if (scanf("%d", &src) != 1) return 0;

dijkstra(graph, src, N);

return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

Anu is given a weighted, directed, and connected graph with V vertices and E edges represented as an adjacency list. They are also given a source vertex S .

The task is to find the shortest distance from the source vertex S to all other vertices in the graph using Dijkstra's algorithm.

Input Format

The first line contains an integer V representing the number of vertices in the graph.

The second line contains an integer E representing the number of edges in the graph.

The next E lines contain three integers each: from, to, and weight, denoting an edge between vertices from and to with the given weight.

The last line contains an integer S representing the source vertex.

Output Format

The output prints a single line in the format:

Shortest distances from vertex S to all other vertices: d0 d1 d2 ... dV-1

where S is the source vertex number and d0, d1, ..., dV-1 are the shortest distances from S to vertices 0 through V-1 respectively, each followed by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 2

1

0 1 9

0

Output: Shortest distances from vertex 0 to all other vertices: 0 9

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>
#include <stdbool.h>
```

```
#define MAX_V 101
```

```
void findShortestPaths() {
    int V, E, S;
```

```
    if (scanf("%d", &V) != 1) return;
    if (scanf("%d", &E) != 1) return;
```

```
    int adj[MAX_V][MAX_V];
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            if (i == j) adj[i][j] = 0;
            else adj[i][j] = -1;
        }
    }
}
```

```
for (int i = 0; i < E; i++) {
    int u, v, w;
```

```

    if (scanf("%d %d %d", &u, &v, &w) == 3) {
        if (adj[u][v] == -1 || w < adj[u][v]) {
            adj[u][v] = w;
        }
    }
}

```

```

if (scanf("%d", &S) != 1) return;

```

```

int dist[MAX_V];
bool visited[MAX_V];

```

```

for (int i = 0; i < V; i++) {
    dist[i] = INT_MAX;
    visited[i] = false;
}

```

```

dist[S] = 0;

```

```

for (int count = 0; count < V; count++) {
    int u = -1;
    int min_dist = INT_MAX;

```

```

    for (int i = 0; i < V; i++) {
        if (!visited[i] && dist[i] < min_dist) {
            min_dist = dist[i];
            u = i;
        }
    }
}

```

```

if (u == -1) break;

```

```

visited[u] = true;

```

```

for (int v = 0; v < V; v++) {
    if (!visited[v] && adj[u][v] != -1 && u != v) {
        if (dist[u] != INT_MAX && dist[u] + adj[u][v] < dist[v]) {
            dist[v] = dist[u] + adj[u][v];
        }
    }
}
}
}

```

```
printf("Shortest distances from vertex %d to all other vertices:", S);  
for (int i = 0; i < V; i++) {  
    printf(" %d", dist[i]);  
}  
printf("\n");  
}  
  
int main() {  
    findShortestPaths();  
    return 0;  
}
```

Status : Partially correct

Marks : 2/10